

Template (1)

template.cpp	22 lines
<pre>#include <bits/stdc++.h> using namespace std; #define endl '\n' #define int long long #define rep(i, a, b) for(int i = a; i < (b); ++i) #define all(x) begin(x), end(x) #define sz(x) ((int)((x).size())) typedef long long ll; typedef pair<int, int> pii; typedef vector<int> vi; void solve() { } signed main() { ios::sync_with_stdio(false); cin.tie(0); cout.tie(0); int t = 1; cin >> t; while (t--) solve(); }</pre>	

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change null_type. To get a multiset, change less to less_equal

<ext/pb.ds/assoc.container.hpp>, <ext/pb.ds/tree.policy.hpp>	819d08, 15 lines
--	------------------

<pre>using namespace __gnu_pbds; template<class T> using Tree = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>; void example() { Tree<int> t, t2; t.insert(8); auto it = t.insert(10).first; assert(it == t.lower_bound(9)); assert(t.order_of_key(10) == 1); assert(t.order_of_key(11) == 2); assert(*t.find_by_order(0) == 8); t.join(t2); // assuming T < T2 or T > T2, merge t2 into t }</pre>	
---	--

SegmentTree.h

Description: 1D point update and range query where cmb is any associative operation. seg[1]==query(0,N-1).

Time: $\mathcal{O}(\log N)$	e03cc5, 20 lines
-----------------------------	------------------

<pre>template<typename T> struct SegTree { // cmb(ID,b) = b T ID; T (*cmb)(T a, T b); int n; vector<T> seg; SegTree(int _n, T id, T _cmb(T, T)) { ID = id; cmb = _cmb; for (n = 1; n < _n;) n *= 2; seg.assign(2*n,ID); } void pull(int p) { seg[p] = cmb(seg[2*p],seg[2*p+1]); } void upd(int p, T val) { // set val at position p seg[p += n] = val; for (p /= 2; p; p /= 2) pull(p); } T query(int l, int r) { // zero-indexed, inclusive T ra = ID, rb = ID; for (l += n, r += n+1; l < r; l /= 2, r /= 2) {</pre>	
---	--

<pre> if (l&l) ra = cmb(ra,seg[l++]); if (r&l) rb = cmb(seg[--r],rb); } return cmb(ra,rb); }</pre>	
---	--

LazySegmentTree.h

Description: 1D range increment and sum query.

Usage: Node* tr = new Node(v, 0, sz(v));	
Time: $\mathcal{O}(\log N)$.	71909a, 28 lines

<pre>template<class T, class U, int SZ> struct LazySeg { // static_assert(pct(SZ) == 1); // SZ must be power of 2 const T ID1; T (*cmb)(T a, T b); vector<T> seg; vector<U> lazy; void (*push)(int,int,int,vector<T>&,vector<U>&) = [&](int ind,int L,int R,vector<T>&seg,vector<U>&lazy) { seg[ind] += (R-L+1)*lazy[ind]; // dependent on operation if (L != R) {lazy[2*ind] += lazy[ind]; lazy[2*ind+1] += lazy[ind];} lazy[ind] = 0; }; // recalc values for current node LazySeg(T id1, U id2, T _cmb(T, T), void _push(int,int,int, vector<T>&,vector<U>&)) : ID1(id1),cmb(_cmb),push(_push),seg(2*SZ,id1),lazy(2*SZ,id2) {} void pull(int ind){seg[ind]=cmb(seg[2*ind],seg[2*ind+1]);} void build() { for(int i=SZ-1;i>=1;i--) pull(i); } void upd(int lo,int hi,U inc,int ind=1,int L=0, int R=SZ-1) { push(ind,L,R,seg,lazy); if (hi < L R < lo) return; if (lo <= L && R <= hi) { lazy[ind] = inc; push(ind,L,R,seg,lazy); return; } int M = (L+R)/2; upd(lo,hi,inc,2*ind,L,M); upd(lo,hi,inc,2*ind+1,M+1,R); pull(ind); } T query(int lo, int hi, int ind=1, int L=0, int R=SZ-1) { push(ind,L,R,seg,lazy); if (lo > R L > hi) return ID1; if (lo <= L && R <= hi) return seg[ind]; int M = (L+R)/2; return cmb(query(lo,hi,2*ind,L,M), query(lo,hi,2*ind+1,M+1,R)); } }</pre>	
---	--

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st,time() and rollback().

Usage: int t = uf.time(); ...; uf.rollback(t);	
Time: $\mathcal{O}(\log N)$	de4ad0, 21 lines

<pre>struct RollbackUF { vi e; vector<pii> st; RollbackUF(int n) : e(n, -1) {} int size(int x) { return -e[find(x)]; } int find(int x) { return e[x] < 0 ? x : find(e[x]); } int time() { return sz(st); } void rollback(int t) { for (int i = time(); i --> t;) e[st[i].first] = st[i].second; st.resize(t); } bool join(int a, int b) { a = find(a), b = find(b); if (a == b) return false; if (e[a] > e[b]) swap(a, b); st.push_back({a, e[a]}); st.push_back({b, e[b]}); e[a] += e[b]; e[b] = a; return true; } }</pre>	
--	--

<pre> } };</pre>	
---------------------	--

RMQ.h

Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.

Usage: RMQ rmq(values);	
rmq.query(inclusive, inclusive);	
Time: $\mathcal{O}(V \log V + Q)$	f33baf, 16 lines

<pre>template<class T> struct RMQ { vector<vector<T>> jmp; RMQ(const vector<T>& V) : jmp(1, V) { for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) { jmp.emplace_back(sz(V) - pw * 2 + 1); rep(j,0,sz(jmp[k])) jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]); } } T query(int a, int b) { b++; // (for right inclusive) assert(a < b); // or return inf if a == b int dep = 31 - __builtin_clz(b - a); return min(jmp[dep][a], jmp[dep][b - (1 << dep)]); } };</pre>	
--	--

LineContainer.h

Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”).

Time: $\mathcal{O}(\log N)$	8ec1c7, 30 lines
-----------------------------	------------------

<pre>struct Line { mutable ll k, m, p; bool operator<(const Line& o) const { return k < o.k; } bool operator<(ll x) const { return p < x; } }; struct LineContainer : multiset<Line, less<>> { // (for doubles, use inf = 1/.0, div(a,b) = a/b) static const ll inf = LLONG_MAX; ll div(ll a, ll b) { // floored division return a / b - ((a ^ b) < 0 && a % b); } bool isect(iterator x, iterator y) { if (y == end()) return x->p = inf, 0; if (x->k == y->k) x->p = x->m > y->m ? inf : -inf; else x->p = div(y->m - x->m, x->k - y->k); return x->p >= y->p; } void add(ll k, ll m) { auto z = insert({k, m, 0}), y = z++, x = y; while (isect(y, z)) z = erase(z); if (x != begin() && isect(--x, y)) isect(x, y = erase(y)); while ((y = x) != begin() && (--x)->p >= y->p) isect(x, erase(y)); } ll query(ll x) { assert(!empty()); auto l = *lower_bound(x); return l.k * x + l.m; } };</pre>	
---	--

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.

Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow	bbbd8f, 11 lines
---	------------------

typedef unsigned long long ull;	
---------------------------------	--

```
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (1l)M);
}

ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \bmod c$.

"ModMu1LL.h"	60dcd1, 12 lines
--------------	------------------

```
bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
        s = __builtin_ctzll(n-1), d = n >> s;
    for (ull a : A) { // ^ count trailing zeroes
        ull p = modpow(a%n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = modmul(p, p, n);
        if (p != n-1 && i != s) return 0;
    }
    return 1;
}
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMu1LL.h", "MillerRabin.h"	d8d98d, 18 lines
-------------------------------	------------------

```
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}

vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

euclid.h

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `__gcd` instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

	33ba8f, 5 lines
--	-----------------

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

CRT.h

Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n, x$ will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.

"euclid.h"	04d93a, 7 lines
------------	-----------------

```
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
```

phiFunction.h

Description: *Euler's* ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p$ prime $\Rightarrow \phi(p^k) = (p - 1)p^{k-1}, m, n$ coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1}p_2^{k_2}...p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1}...(p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$.
 $\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, \gcd(k, n) = 1} k = n\phi(n)/2, n > 1$
Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \forall a$.

	cf7d6d, 8 lines
--	-----------------

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
    rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
    for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
        for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

1.1 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$$\sum_{d|n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$$

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call `setpi` before `maxflow`, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}(FE \log(V))$ where F is max flow. $\mathcal{O}(VE)$ for `setpi`.

#include <bits/extc++.h>	4b9fe1, 79 lines
--------------------------	------------------

```
const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
};
```

```
int N;
vector<vector<edge>> ed;
vi seen;
vector<ll> dist, pi;
vector<edge*> par;

MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}
```

```
void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    ed[from].push_back(edge{ from,to, (int)sz(ed[to]),cap,cost,0
    });
    ed[to].push_back(edge{ to,from, (int)sz(ed[from])-1,0,-cost
    ,0 });
}
```

```
void path(int s) {
    fill(all(seen), 0);
    fill(all(dist), INF);
    dist[s] = 0; ll di;

    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({ 0, s });
```

```
    while (!q.empty()) {
        s = q.top().second; q.pop();
        seen[s] = 1; di = dist[s] + pi[s];
        for (edge& e : ed[s]) if (!seen[e.to]) {
            ll val = di - pi[e.to] + e.cost;
            if (e.cap - e.flow > 0 && val < dist[e.to]) {
                dist[e.to] = val;
                par[e.to] = &e;
                if (its[e.to] == q.end())
                    its[e.to] = q.push({ -dist[e.to], e.to });
                else
                    q.modify(its[e.to], { -dist[e.to], e.to });
            }
        }
        rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
    }

    pair<ll, ll> maxflow(int s, int t) {
        ll totflow = 0, totcost = 0;
        while (path(s), seen[t]) {
            ll fl = INF;
            for (edge* x = par[t]; x; x = par[x->from])
                fl = min(fl, x->cap - x->flow);

            totflow += fl;
            for (edge* x = par[t]; x; x = par[x->from]) {
                x->flow += fl;
                ed[x->to][x->rev].flow -= fl;
            }
        }
        rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
        return {totflow, totcost/2};
    }
}
```

```
// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
        rep(i,0,N) if (pi[i] != INF)
            for (edge& e : ed[i]) if (e.cap)
                if ((v = pi[i] + e.cost) < pi[e.to])
                    pi[e.to] = v, ch = 1;
```

```
    assert(it >= 0); // negative cost cycle
}
};
```

Dinic.h

Description: Flow algorithm with complexity $O(VE\log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{VE})$ for bipartite matching.

d7f0f1, 42 lines

```
struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0; q[0] = s;
        rep(L, 0, 31) do { // 'int L=30' maybe faster for random data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
            while (lvl[p = dfs(s, t, LLONG_MAX)] flow += p;
        } while (lvl[t]);
        return flow;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: vi btoa(m, -1); dfsMatching(g, btoa);
Time: $O(VE)$

522b98, 22 lines

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
    if (btoa[j] == -1) return 1;
    vis[j] = 1; int di = btoa[j];
    for (int e : g[di])
        if (!vis[e] && find(e, g, btoa, vis)) {
            btoa[e] = di;
            return 1;
        }
    return 0;
}
```

```
}
int dfsMatching(vector<vi>& g, vi& btoa) {
    vi vis;
    rep(i, 0, sz(g)) {
        vis.assign(sz(btoa), 0);
        for (int j : g[i])
            if (find(j, g, btoa, vis)) {
                btoa[j] = i;
                break;
            }
    }
    return sz(btoa) - (int)count(all(btoa), -1);
}
```

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.
Usage: scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.
Time: $O(E + V)$

76b5c9, 24 lines

```
vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
    int low = val[j] = ++Time, x; z.push_back(j);
    for (auto e : g[j]) if (comp[e] < 0)
        low = min(low, val[e] ?: dfs(e, g, f));

    if (low == val[j]) {
        do {
            x = z.back(); z.pop_back();
            comp[x] = ncomps;
            cont.push_back(x);
        } while (x != j);
        f(cont); cont.clear();
        ncomps++;
    }
    return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
    int n = sz(g);
    val.assign(n, 0); comp.assign(n, -1);
    Time = ncomps = 0;
    rep(i, 0, n) if (comp[i] < 0) dfs(i, g, f);
}
```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.
Time: $O(N\log N + Q)$

"../data-structures/RMQ.h"

d9fc67, 21 lines

```
struct LCA {
    int T = 0;
    vi time, path, ret;
    RMQ<int> rmq;

    LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C, 0, -1), ret)) {}
    void dfs(vector<vi>& C, int v, int par) {
        time[v] = T++;
        for (int y : C[v]) if (y != par) {
            path.push_back(v), ret.push_back(time[v]);
            dfs(C, y, v);
        }
    }
}
```

```
int lca(int a, int b) {
    if (a == b) return a;
    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b - 1)];
}
//dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
```

Centroid.h

Description: Find centroid decomposition of a tree. Main function to be called is solve(root, root, [](ll v)). Process trees formed after decomposition at each step within process(node)
Time: $O(N\log N)$

cb3bfe, 35 lines

```
struct Centroid { // 0 based indexing.
    vector<vi> centroid_tree, adj;
    vi par_ct, siz;
    vector<bool> vis;
    Centroid(vector<vector<ll>> _adj) : adj(_adj), cent_tree(sz(_adj)),
        siz(sz(_adj)), vis(sz(_adj)), par_ct(sz(_adj), -1) {}
    void find_size(ll v, ll par) {
        siz[v] = 1;
        for (auto x : adj[v]) {
            if (x == par || vis[x]) continue;
            find_size(x, v);
            siz[v] += siz[x];
        }
    }
    ll find_centroid(ll v, ll par, ll n_nodes) {
        for (auto x : adj[v]) {
            if (x == par || vis[x]) continue;
            if (siz[x] > n_nodes/2) return find_centroid(x, v, n_nodes);
        }
        return v;
    }
    void process(ll v) {} // optional
    ll solve(ll v, ll prev) {
        find_size(v, prev);
        ll centroid = find_centroid(v, prev, siz[v]);
        vis[centroid] = true;
        par_ct[centroid] = prev;
        process(centroid);
        for (auto x : adj[centroid]) {
            if (vis[x]) continue;
            centroid_tree[centroid].push_back(solve(x, centroid));
        }
        return centroid;
    }
};
```

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.
Time: $O(n)$

d4375c, 16 lines

```
vi pi(const string& s) {
    vi p(sz(s));
    rep(i, 1, sz(s)) {
        int g = p[i-1];
        while (g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}

vi match(const string& s, const string& pat) {
```

```
vi p = pi(pat + '\0' + s), res;
rep(i,sz(p)-sz(s),sz(p))
    if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
return res;
}
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$

	ee09e2, 12 lines
<pre>vi Z(const string& S) { vi z(sz(S)); int l = -1, r = -1; rep(i,l,sz(S)) { z[i] = i >= r ? 0 : min(r - i, z[i - l]); while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]]) z[i]++; if (i + z[i] > r) l = i, r = i + z[i]; } return z; }</pre>	

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).

	e7ad79, 13 lines
<pre>array<vi, 2> manacher(const string& s) { int n = sz(s); array<vi,2> p = {vi(n+1), vi(n)}; rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) { int t = r-i+!z; if (i<r) p[z][i] = min(t, p[z][l+t]); int L = i-p[z][i], R = i+p[z][i]-!z; while (L>=1 && R+1<n && s[L-1] == s[R+1]) p[z][i]++, L--, R++; if (R>r) l=L, r=R; } return p; }</pre>	

SuffixArray.h

Description: Builds suffix array for a string. sa[i] is the starting index of the suffix which is i 'th in the sorted suffix array. The returned vector is of size $n + 1$, and sa[0] = n. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any nul chars.
Time: $\mathcal{O}(n \log n)$

	635552, 22 lines
<pre>struct SuffixArray { vi sa, lcp; SuffixArray(string s, int lim=256) { // or vector<int> s.push_back(0); int n = sz(s), k = 0, a, b; vi x(all(s)), y(n), ws(max(n, lim)); sa = lcp = y, iota(all(sa), 0); for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) { p = j, iota(all(y), n - j); rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j; fill(all(ws), 0); rep(i,0,n) ws[x[i]]++; rep(i,l,lim) ws[i] += ws[i - 1]; for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i]; swap(x, y), p = 1, x[sa[0]] = 0; rep(i,l,n) a = sa[i - 1], b = sa[i], x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++; } } };</pre>	

```
for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
    for (k && k--, j = sa[x[i] - 1];
         s[i + k] == s[j + k]; k++);
}
};
```

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

	47ec0a, 28 lines
<pre>template <class T> int sgn(T x) { return (x > 0) - (x < 0); } template<class T> struct Point { typedef Point P; T x, y; explicit Point(T x=0, T y=0) : x(x), y(y) {} bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); } bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); } P operator+(P p) const { return P(x+p.x, y+p.y); } P operator-(P p) const { return P(x-p.x, y-p.y); } P operator*(T d) const { return P(x*d, y*d); } P operator/(T d) const { return P(x/d, y/d); } T dot(P p) const { return x*p.x + y*p.y; } T cross(P p) const { return x*p.y - y*p.x; } T cross(P a, P b) const { return (a-*this).cross(b-*this); } T dist2() const { return x*x + y*y; } double dist() const { return sqrt((double)dist2()); } // angle to x-axis in interval [-pi, pi] double angle() const { return atan2(y, x); } P unit() const { return *this/dist(); } // makes dist()==1 P perp() const { return P(-y, x); } // rotates +90 degrees P normal() const { return perp().unit(); } // returns point rotated 'a' radians ccw around the origin P rotate(double a) const { return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); } friend ostream& operator<<(ostream& os, P p) { return os << "(" << p.x << ", " << p.y << ")"; } };</pre>	

lineDistance.h

Description: Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

	f6bf6b, 4 lines
<pre>template<class P> double lineDist(const P& a, const P& b, const P& p) { return (double) (b-a).cross(p-a) / (b-a).dist(); }</pre>	

SegmentDistance.h

Description: Returns the shortest distance between point p and the line segment from point s to e.
Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;

	5c88f4, 6 lines
<pre>"Point.h" typedef Point<double> P; double segDist(P& s, P& e, P& p) { if (s==e) return (p-s).dist(); auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s))); return ((p-s)*d-(e-s)*t).dist()/d; }</pre>	

SegmentIntersection.h

Description: If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.
Usage: vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
 cout << "segments intersect at " << inter[0] << endl;
"Point.h", "OnSegment.h"9d5f2f, 13 lines

<pre>template<class P> vector<P> segInter(P a, P b, P c, P d) { auto oa = c.cross(d, a), ob = c.cross(d, b), oc = a.cross(b, c), od = a.cross(b, d); // Checks if intersection is single non-endpoint point. if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0) return {(a * ob - b * oa) / (ob - oa)}; set<P> s; if (onSegment(c, d, a)) s.insert(a); if (onSegment(c, d, b)) s.insert(b); if (onSegment(a, b, c)) s.insert(c); if (onSegment(a, b, d)) s.insert(d); return {all(s)}; }</pre>	
---	--

sideOf.h

Description: Returns where p is as seen from s towards e . $1/0/-1 \Leftrightarrow$ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.
Usage: bool left = sideOf(p1,p2,q)==1;

	3af81c, 9 lines
<pre>"Point.h" template<class P> int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }</pre>	

OnSegment.h

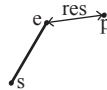
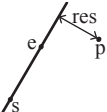
Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

	c597e8, 3 lines
<pre>"Point.h" template<class P> bool onSegment(P s, P e, P p) { return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0; }</pre>	

ConvexHull.h

Description: Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.
Time: $\mathcal{O}(n \log n)$

	310954, 13 lines
<pre>"Point.h" typedef Point<ll> P; vector<P> convexHull(vector<P> pts) { if (sz(pts) <= 1) return pts; sort(all(pts)); vector<P> h(sz(pts)+1); int s = 0, t = 0;</pre>	



```
for (int it = 2; it--; s = --t, reverse(all(pts)))
for (P p : pts) {
    while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
    h[t++] = p;
}
return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

InsidePolygon.h
Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.
Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};
bool in = inPolygon(v, P{3, 3}, false);
Time: $\mathcal{O}(n)$

```
"Point.h", "OnSegment.h", "SegmentDistance.h"                                2bf504, 11 lines

template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
    int cnt = 0, n = sz(p);
    rep(i,0,n) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a) return !strict;
        //or: if (segDist(p[i], q, a) <= eps) return !strict;
        cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
    }
    return cnt;
}
```

PolygonArea.h
Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!
Time: $\mathcal{O}(n)$

```
"Point.h"                                                                    f12300, 6 lines

template<class T>
T polygonArea2(vector<Point<T>>& v) {
    T a = v.back().cross(v[0]);
    rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
    return a;
}
```

PointInsideHull.h
Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}(\log N)$

```
"Point.h", "sideOf.h", "OnSegment.h"                                         71446b, 14 lines

typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
    int a = 1, b = sz(l) - 1, r = !strict;
    if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <= -r) return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

FastFourierTransform.h
Description: fft(a) computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: conv(a, b) = c, where $c[x] = \sum a[i]b[x - i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod.

```
Time: O(N log N) with N = |A| + |B| (~1s for N = 2^22)                                00ced6, 35 lines

typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0/L, acos(-1.0/L) / k);
        rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
    }
vd conv(const vd& a, const vd& b) {
    if (a.empty() || b.empty()) return {};
    vd res(sz(a) + sz(b) - 1);
    int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
    vector<C> in(n), out(n);
    copy(all(a), begin(in));
    rep(i,0,sz(b)) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
    rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
    return res;
}
```

FastFourierTransformMod.h
Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in [0, mod).
Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

```
"FastFourierTransform.h"                                                       cad46c, 22 lines

typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    vl res(sz(a) + sz(b) - 1);
    int B=32-__builtin_clz(sz(res)), n=1<<B, cut=(int)(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    rep(i,0,n) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl), fft(outs);
    rep(i,0,sz(res)) {
        ll av = (ll)(real(outl[i])+.5), cv = (ll)(imag(outs[i])+.5);
        ll bv = (ll)(imag(outl[i])+.5) + (ll)(real(outs[i])+.5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}
```

NumberTheoreticTransform.h
Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(\text{mod}-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a . For arbitrary modulo, see FFTMod. conv(a, b) = c, where $c[x] = \sum a[i]b[x - i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in [0, mod).
Time: $\mathcal{O}(N \log N)$

```
"../number-theory/ModPow.h"                                                    ced03d, 35 lines

const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll> vl;
void ntt(vl &a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vl rt(2, 1);
    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        ll z[] = {1, modpow(root, mod >> s)};
        rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
            a[i + j + k] = ai - z + (z > ai ? mod : 0);
            ai += (ai + z >= mod ? z - mod : z);
        }
    }
vl conv(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
        n = 1 << B;
    int inv = modpow(n, mod - 2);
    vl L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i,0,n)
        out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
}
```

- 1.1.1 Bit hacks
 - $x \ \& \ -x$ is the least bit in x .
 - for (int x = m; x;) { --x &= m; ... } loops over all subset masks of m (except m itself).
 - $c = x \& -x, \ r = x + c; ((r \wedge x) \gg 2) / c \mid r$ is the next number after x with the same number of bits set.
 - rep(b,0,K) rep(i,0,(1 << K)) if (i & 1 << b) D[i] += D[i^(1 << b)]; computes all sums of subsets.
 - for (int i = b._Find_first(); i < b.size(); i = b._Find_next(i)) can be used to iterate through the set bits of a bitset.